

ACADEVIA

Smart India Hackathon — Technical Documentation

Project Name	Acadevia — Gamified Learning Platform for Bharat
Problem Statement ID	SIH 2026 — Education / EdTech (Rural & Tier-2/3 focus)
Category	Software • Full-Stack • Cloud-Native Microservices
Architecture	16 Spring Boot Microservices + React 19 PWA on Kubernetes
Primary Languages	Java 17 (backend), TypeScript 5 (frontend)
Submission Date	15 August 2026
Team	Team Acadevia

This document provides the complete technical specification, tech stack, service-to-service connections, request lifecycle, deployment topology, and infrastructure blueprint for the Acadevia learning platform, prepared for Smart India Hackathon jury evaluation.

1. Executive Summary

Acadevia is India's largest gamified, offline-first, multilingual learning platform designed for K-12 students, especially in low-bandwidth and rural areas. The platform is built as a cloud-native, event-driven system comprising **16 independent Spring Boot microservices**, a **React 19 + Vite PWA** front-end, and a fully automated **Kubernetes + Jenkins CI/CD** pipeline. The system supports JWT-based authentication, role-based access control, real-time leaderboards (WebSocket), Kafka-driven async communication, Redis-backed rate limiting, full observability (Prometheus, Grafana, Zipkin, Loki), and offline learning via Dexie + Workbox Service Workers.

1.1 Key Capabilities

- ■ Gamified learning with XP, badges, streaks, and live leaderboards
- ■ Offline-first PWA — works on 2G, syncs when online
- ■ Multi-language (i18n) support — English, Hindi, regional
- ■ JWT-based stateless auth with refresh tokens
- ■ Video & interactive content delivery (HLS-ready)
- ■ Real-time analytics for teachers and admins
- ■ AI-personalized quiz recommendations
- ■ Multi-tenant school & classroom management
- ■■ Cloud-native — runs on any Kubernetes cluster

2. Complete Technology Stack

2.1 Frontend Stack (acadevia-frontend)

Layer	Technology	Version	Purpose
Framework	React	19.2.0	Component-based UI library
Language	TypeScript	5.9.3	Type-safe JavaScript
Build Tool	Vite	7.3.1	Lightning-fast dev server & bundler
Routing	React Router DOM	7.13.0	Client-side routing
Server State	TanStack React Query	5.90.21	API caching & data fetching
Client State	Zustand	5.0.11	Lightweight state management
Forms	React Hook Form + Zod	7.71 / 4.3	Form validation with schema
UI Animations	Framer Motion	12.34	Smooth, GPU-accelerated animations
Styling	Tailwind CSS + CVA	3.4 / 0.7	Utility-first + design system
Icons	Lucide React	0.564	Tree-shakable SVG icons
Charts	Recharts	3.7	Data visualization
Realtime	Socket.io-client	4.8	WebSocket connections
Offline DB	Dexie	4.3	IndexedDB wrapper for offline cache
PWA	Vite-Plugin-PWA + Workbox	7.4	Service worker, manifest, installable
Audio	Howler	2.2	Game sounds & audio cues
i18n	i18next + react-i18next	25.8 / 16.5	Multi-language support
HTTP Client	Axios	1.13	Promise-based REST client
Video Player	React Player	3.4	HLS / YouTube / file playback
Toasts	Sonner	2.0	Non-blocking notifications
Date Utils	date-fns	4.1	Lightweight date manipulation
Linting	ESLint + typescript-eslint	9.39 / 8.48	Code quality
Testing	Vitest + Testing Library	4.0 / 16.3	Unit & integration tests
E2E	Playwright	latest	Browser automation tests

2.2 Backend Stack (Spring Boot Microservices)

Component	Technology	Version	Purpose
Language	Java	17 (LTS)	Modern, performant JVM language
Framework	Spring Boot	3.2.3	Production-grade microservices
Service Mesh	Spring Cloud	2023.0.0	Eureka, Config, Gateway, OpenFeign

Component	Technology	Version	Purpose
API Gateway	Spring Cloud Gateway	3.2.x	Reactive WebFlux gateway
Service Discovery	Netflix Eureka	4.x	Dynamic service registration
Central Config	Spring Cloud Config Server	4.x	Externalized configuration
Security	Spring Security + JWT	6.x / 0.12.5	Auth & JWT signing
Persistence	Spring Data JPA + Hibernate	3.2 / 6.4	ORM & repository pattern
DB Migrations	Flyway (core + mysql)	10.x	Versioned DB schema migrations
Mapping	MapStruct + ModelMapper	1.5.5 / 3.2	DTO ↔ Entity conversion
Messaging	Spring Kafka	3.1.x	Async event-driven comms
Realtime	WebSocket (STOMP)	—	Live leaderboard & notifications
Rate Limiting	Redis (reactive)	7-alpine	Token-bucket via gateway
Caching	Redis	7-alpine	Distributed cache, sessions
Boilerplate Reduction	Lombok	1.18.32	Annotations for code gen
API Docs	springdoc-openapi	2.3.0	OpenAPI 3 / Swagger UI
Build	Maven	3.9+	Multi-module build system
Testing	JUnit 5 + Mockito + H2	—	Unit & integration testing

2.3 Data Layer

Database	Engine	Version	Used By
Primary OLTP	MySQL	8.0	All 16 microservices (logical DB per service)
Cache	Redis	7-alpine	API Gateway, sessions, rate-limit, leaderboard
Search (planned)	Elasticsearch	8.x	Content / course search
Object Storage	MinIO (S3-compatible)	latest	Videos, thumbnails, attachments
Migrations	Flyway	10.x	Schema versioning

2.4 Infrastructure & DevOps

Layer	Tool	Purpose
Containerization	Docker	Immutable service images
Orchestration	Kubernetes	Production-grade container orchestration
Service Mesh (opt)	Istio / Spring Cloud	mTLS, traffic shaping
CI/CD	Jenkins (declarative pipeline)	Build → Test → Scan → Deploy
Container Registry	Docker Registry / Harbor	Image versioning

Layer	Tool	Purpose
Config Mgmt	Spring Cloud Config Server (native)	Centralized config
Service Discovery	Eureka	Health-checked registry
API Gateway	Spring Cloud Gateway (WebFlux)	Routing, JWT, rate-limit
Tracing	Zipkin + Micrometer	Distributed request tracing
Metrics	Prometheus	Time-series metrics scraping
Visualization	Grafana	Operational dashboards
Log Aggregation	Loki + Promtail	Centralized logs
Secret Mgmt	Kubernetes Secrets / Vault (opt)	Credential handling
Build Automation	Makefile (academia-infrastructure/)	Local orchestration
IaC	Kubernetes YAML (kustomize-ready)	Declarative infra

2.5 Security Stack

Concern	Implementation
Authentication	JWT (HS256) with access (15 min) + refresh (7 day) tokens
Authorization	Role-based (STUDENT, TEACHER, SCHOOL_ADMIN, ADMIN)
Password Hashing	BCrypt (Spring Security default, cost factor 10)
Transport Security	TLS 1.2+ at ingress; mTLS between pods (opt)
API Protection	Redis-backed token-bucket rate limiting at gateway
CORS	Whitelisted origins in gateway
Input Validation	Jakarta Bean Validation + Zod on frontend
Vulnerability Scan	Trivy (image), OWASP Dependency-Check (Jenkins)
Secret Mgmt	Env vars / k8s secrets; JWT_SECRET rotated via config server

2.6 Observability Stack

Signal	Tool	Where
Metrics	Micrometer → Prometheus	Each Spring Boot service exposes /actuator/prometheus
Dashboards	Grafana	Connected to Prometheus + Loki
Tracing	Zipkin	Across gateway → services → DB
Logs	Loki + Promtail (k8s) or Logback (local)	Aggregated, searchable
Health Checks	Spring Actuator	/actuator/health, /actuator/info
Uptime	Eureka heartbeat (30s)	Auto-removal of unhealthy instances

3. System Architecture Overview

Acadevia follows the **Microfrontends-friendly, API-first, Cloud-Native, Event-Driven Microservices** pattern. All services are independently deployable, own their data (Database-per-Service), and communicate over a mix of **synchronous REST** (via Eureka + OpenFeign) and **asynchronous Kafka events**.

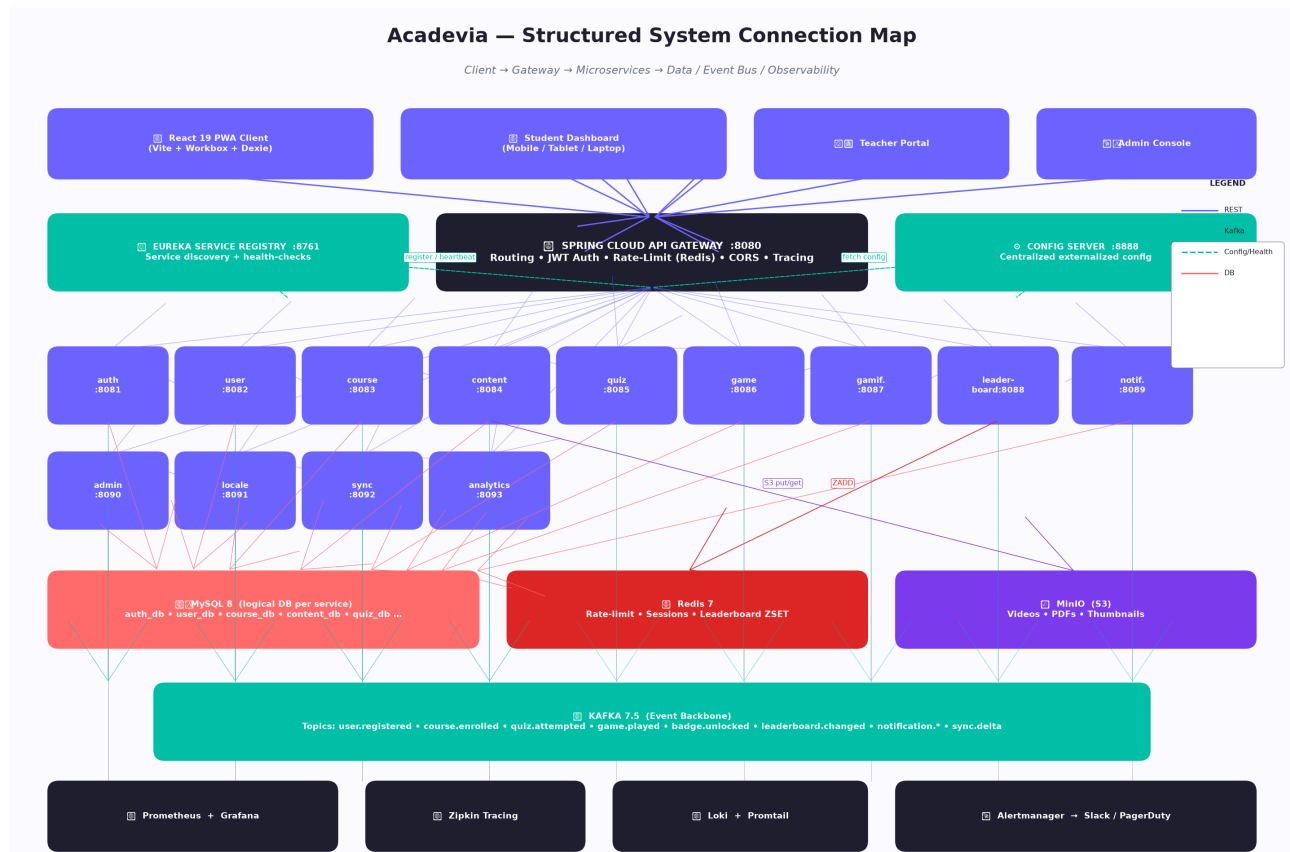


Figure 1 — Structured system connection map showing client → gateway → microservices → data / event-bus / observability layers.

3.1 High-Level Architecture (Textual Blueprint)

```
■ CLIENTS (PWA / Mobile Web) ■  
■ React 19 + Vite + Workbox • Offline (Dexie/IndexedDB) ■  
  
■ HTTPS / WSS  
▼  
  
■ SPRING CLOUD API GATEWAY :8080 ■  
■ • Routing (path-based) • JWT validation • Rate-limit (Redis)  
■ • CORS • Load-balancing (lb://) • Logging • Tracing ■  
  
■ ■ ■ ■ ■  
▼ ▼ ▼ ▼ ▼  
  
■ AUTH ■ ■ USER ■ ■ COURSE ■ ■ CONTENT ■ ■ QUIZ ■ ... 16  
■ :8081 ■ ■ :8082 ■ ■ :8083 ■ ■ :8084 ■ ■ :8085 ■ services  
  
■ ■ ■ ■ ■  
▼ ▼ ▼ ▼ ▼  
  
■ EUREKA SERVICE REGISTRY :8761 (heartbeat 30s) ■  
  
■ ■ ■ ■ ■  
▼ ▼ ▼ ▼ ▼  
  
■ SPRING CLOUD CONFIG SERVER :8888 (native file backend) ■  
  
■ ■ ■ ■ ■  
▼ ▼ ▼ ▼ ▼  
  
■ MySQL 8 ■ ■ MySQL 8 ■ ■ MySQL ■ ■ MySQL 8 ■ ■ MySQL 8 ■ log  
■ auth_db ■ ■ user_db ■ ■ course ■ ■ content ■ ■ quiz_db ■ per  
  
■ ■ ■ ■ ■  
▼ ▼ ▼ ▼ ▼  
  
■ KAFKA 7.5 (event bus) ■ ■ REDIS 7 (cache + RL) ■  
■ topics: user.events, ■ ■ • Rate-limit tokens ■  
■ quiz.events, gamif.*, ■ ■ • Session cache ■  
■ leaderboard.*, notif.* ■ ■ • Leaderboard ZSET ■  
  
■ OBSERVABILITY • Prometheus • Grafana • Zipkin • Loki ■
```

4. Microservice Catalog

All 16 services are Spring Boot 3.2.3 Java 17 Maven modules. They share a common parent POM that pins Spring Cloud 2023.0.0, Lombok 1.18.32, MapStruct 1.5.5, and JWT 0.12.5.

#	Service	Port	DB	Key Responsibility
1	config-server	8888	—	Externalized config (native FS)
2	service-registry	8761	—	Eureka server for discovery
3	api-gateway	8080	Redis	Reactive gateway: routing, JWT, rate-limit
4	auth-service	8081	auth_db	Register, login, JWT, refresh, OAuth
5	user-service	8082	user_db	Profile, geography, schools, classrooms
6	course-service	8083	course_db	Courses, lessons, enrollments
7	content-service	8084	content_db	Videos, PDFs, thumbnails (MinIO)
8	quiz-service	8085	quiz_db	Quizzes, attempts, scoring
9	game-service	8086	game_db	Mini-games, puzzles, scenarios
10	gamification-service	8087	gamif_db	XP, badges, streaks, levels
11	leaderboard-service	8088	Redis	Real-time ranking (WebSocket + ZSET)
12	notification-service	8089	notif_db	Email, push, in-app notifications
13	admin-service	8090	admin_db	Admin dashboards, moderation
14	locale-service	8091	locale_db	Translations, i18n bundles
15	sync-service	8092	sync_db	Offline delta sync reconciliation
16	analytics-service	8093	analytics	Aggregations, KPIs, ML features

5. Structured System Connection Map

5.1 Synchronous (REST) Connections

All inter-service REST calls go through the API Gateway. Services use **OpenFeign clients** with **Eureka service IDs** for resolution. Below is the route table configured in *api-gateway/application.yml*.

Gateway Path	Target Service (lb://)	Rate-Limit (rps / burst)	Auth
/api/v1/auth/**	AUTH-SERVICE	10 / 20	Optional (issues JWT)
/api/v1/users/**	USER-SERVICE	100 / 100	JWT
/api/v1/geography/**	USER-SERVICE	100 / 100	JWT
/api/v1/schools/**	USER-SERVICE	100 / 100	JWT
/api/v1/classrooms/**	USER-SERVICE	100 / 100	JWT
/api/v1/courses/**	COURSE-SERVICE	100 / 100	JWT
/api/v1/enrollments/**	COURSE-SERVICE	100 / 100	JWT
/api/v1/lessons/**	COURSE-SERVICE	100 / 100	JWT
/api/v1/content/**	CONTENT-SERVICE	100 / 100	JWT
/api/v1/videos/**	CONTENT-SERVICE	100 / 100	JWT
/api/v1/quizzes/**	QUIZ-SERVICE	100 / 100	JWT
/api/v1/games/**	GAME-SERVICE	100 / 100	JWT
/api/v1/gamification/**	GAMIFICATION-SERVICE	100 / 100	JWT
/api/v1/leaderboard/**	LEADERBOARD-SERVICE	100 / 100	JWT
/api/v1/notifications/**	NOTIFICATION-SERVICE	100 / 100	JWT
/api/v1/analytics/**	ANALYTICS-SERVICE	100 / 100	JWT
/api/v1/admin/**	ADMIN-SERVICE	100 / 100	JWT (Admin)
/api/v1/i18n/**	I18N-SERVICE	100 / 100	JWT
/api/v1/sync/**	SYNC-SERVICE	100 / 100	JWT
/ws/**	LEADERBOARD-SERVICE	100 / 100	Token in query

5.2 Asynchronous (Kafka) Connections

Kafka is the event backbone. Services publish **domain events** after DB commits; downstream services consume them to update projections, trigger side effects, or fan-out notifications.

Topic	Producer	Consumers	Event Payload
user.registered	auth-service	user-service, gamification, notification	{userId, email, role, locale}
user.profile.updated	user-service	gamification, leaderboard	{userId, displayName, avatar}
course.enrolled	course-service	gamification, notification, analytics	{userId, courseId, enrolledAt}

Topic	Producer	Consumers	Event Payload
lesson.completed	course-service	gamification, leaderboard, analytics	{userId, lessonId, courseId}
quiz.attempted	quiz-service	gamification, leaderboard, analytics	{userId, quizId, score, time}
game.played	game-service	gamification, leaderboard	{userId, gameId, score}
badge.unlocked	gamification-service	notification, leaderboard	{userId, badgeId, ts}
leaderboard.changed	leaderboard-service	notification, analytics	{scope, top10, ts}
notification.requested	*	notification-service	{userId, type, payload}
sync.delta	sync-service	course-service, quiz-service, content	{userId, ops[]}

5.3 WebSocket / Realtime Connections

Endpoint	Service	Client Use
/ws/leaderboard	leaderboard-service	Live ranking updates on student dashboard
/ws/notifications	notification-service	In-app toasts & badge unlocks
/ws/game	game-service	Multiplayer turn-based mini-games

5.4 Inter-Service Dependency Matrix

The matrix below summarizes which service directly calls (sync/async) which other service. **R** = REST via Feign, **K** = Kafka event, **W** = WebSocket.

Service	config-server	service-registry	auth-service	user-service	course-service	content-service	quiz-service	game-service	gamification-service	leaderboard-service	notification-service	admin-service	locale-service	sync-service	analytics-service
config-server	—														
service-registry		—													
api-gateway			—	R	R	R	R	R	R	R	R	R	R	R	R
auth-service				—	K				K		K				
user-service				K	—				K	K					
course-service						—			K	K	K				K
content-service							—								
quiz-service								—	K	K					K
game-service									—	K	K				
gamification-service										—	K	K			
leaderboard-service											—	K			K
notification-service												—			
admin-service													—		
locale-service														—	
sync-service															—
analytics-service															—

Legend: R = REST (sync, via Feign) • K = Kafka event (async) • W = WebSocket • — = self (not applicable). *Note: The gateway and registry/config implicitly connect to every service for routing & config; columns left blank in the matrix above to focus on business-level couplings.*

6. End-to-End Request Lifecycle

Example flow — a student submits a quiz attempt and the leaderboard updates in real time:

1. CLIENT (PWA)
 - User taps "Submit Quiz" → optimistic update in React Query cache
 - Axios POST /api/v1/quizzes/{id}/attempts with Bearer JWT
2. API GATEWAY (8080)
 - RequestRateLimiter checks Redis token bucket (100 rps)
 - JwtAuthenticationFilter validates JWT (signature, exp, role)
 - Routes to lb://QUIZ-SERVICE
3. QUIZ-SERVICE (8085)
 - @PreAuthorize checks role = STUDENT
 - Validates payload (Bean Validation)
 - Persists attempt to MySQL quiz_db
 - Publishes QuizAttemptedEvent to Kafka topic quiz.attempted
 - Returns 201 Created with score
4. KAFKA FAN-OUT
 - gamification-service consumes → awards XP, checks badge thresholds, persists, then publishes badge.unlocked (if any)
 - leaderboard-service consumes → ZADD into Redis ZSET, publishes leaderboard.changed
 - analytics-service consumes → updates daily aggregates
5. REALTIME PUSH
 - leaderboard-service WebSocket pushes new top-N to /ws/leaderboard
 - notification-service pushes in-app toast on badge.unlocked
6. CLIENT RECEIVES
 - Socket.io-client updates Zustand store → React re-renders
 - TanStack Query invalidates /leaderboard/me → refetch
 - Sonner toast displays "■ Badge unlocked!"

7. Deployment Topology

7.1 Local Development (Docker Compose)

The *docker/docker-compose.yml* file spins up the complete stack on a single host:

Service	Image	Host Port	Purpose
MySQL 8.0	mysql:8.0	3307	Primary database
Redis 7	redis:7-alpine	6379	Cache + rate-limit
Zookeeper	confluentinc/cp-zookeeper:7.5.0	2181	Kafka coordination
Kafka 7.5	confluentinc/cp-kafka:7.5.0	9092	Event bus
Zipkin	openzipkin/zipkin	9411	Distributed tracing
Prometheus	prom/prometheus	9090	Metrics scraping
Grafana	grafana/grafana	3001	Dashboards
Config Server	build ../config-server	8888	Centralized config
Service Registry	build ../service-registry	8761	Eureka
API Gateway	build ../api-gateway	8080	Single entry point
Auth Service	build ../auth-service	8081	JWT issuance

7.2 Production (Kubernetes)

Production runs on Kubernetes using manifests in *acadevia-infrastructure/kubernetes/*:

Folder	Resources
namespace.yml	Namespace, ResourceQuotas, LimitRanges
infrastructure/	StatefulSets for MySQL, Redis, Kafka, Zookeeper, MinIO
config/	ConfigMaps, Secrets (DB creds, JWT keys)
services/	Deployment + Service + HPA per microservice (16 + frontend)
networking/	Ingress (NGINX), NetworkPolicies, Gateway API
scaling/	HPA, VPA, KEDA scalers (Kafka lag, CPU)
jobs/	One-shot jobs: db-migration, seed-data, badge-recalc
monitoring/	Prometheus, Grafana dashboards, Jaeger, Loki, Promtail
scripts/	Helper bash scripts (apply, rollback, port-forward)

7.3 CI/CD Pipeline (Jenkins)

The *acadevia-infrastructure/ci-cd/Jenkinsfile* implements a 6-stage pipeline:

Stage	Action
1. Build	Parallel: mvn package for all 16 services + npm ci/build for frontend
2. Test	Backend (mvn verify + JaCoCo) and Frontend (lint, tsc, vitest) in parallel
3. Docker Build	Tag & push \${REGISTRY}/\${svc}:\${SHORT_SHA} and :latest
4. Security	Trivy image scan (CRITICAL/HIGH) + OWASP dependency-check
5. Deploy Staging	On develop branch — run DB migration job → rolling update → smoke test
6. Deploy Prod	On main branch — manual approval → rolling update → smoke test → auto-rollback on failure → git tag release

8. Non-Functional Requirements

Requirement	Target	How Achieved
Scalability	1M+ MAU, 50K concurrent users	Stateless services + HPA + Kafka partitioning
Availability	99.9% uptime	K8s multi-replica, health-checks, rolling updates
Performance	API p95 < 300ms	Redis caching, DB indexes, CDN, Workbox precache
Security	OWASP Top-10 compliant	JWT, BCrypt, rate-limit, Trivy, OWASP-Dep-Check
Offline	Full lesson on 2G/edge	Service Worker (Workbox) + Dexie + sync-service
i18n	≥ 5 Indian languages	i18next + locale-service (translation API)
Observability	MTTR < 15 min	Prometheus + Grafana + Zipkin + Loki + alerts
Portability	Runs on any K8s	Docker images, declarative YAML, no vendor lock-in
Cost	Optimized for low infra cost	Scale-to-zero options, HPA, spot nodes (opt)

9. Justification of Technology Choices

Spring Boot 3 + Java 17 — Mature, battle-tested, huge ecosystem. Java 17 LTS ensures long-term support and modern features (records, sealed types, pattern matching).

Microservices (not monolith) — Independent deploys, fault isolation, team autonomy — critical for a 16-module platform with multiple contributors.

Spring Cloud Gateway (WebFlux) — Reactive, non-blocking I/O — handles 10K+ concurrent connections on modest hardware.

Eureka over Consul/K8s-DNS — Spring-native, simple, well-documented, no extra cluster to operate.

MySQL 8 — Most familiar RDBMS for the team; ACID for quizzes/payments; Flyway for safe schema evolution.

Redis 7 — Single tool for cache, rate-limit, sessions, and sorted-set leaderboard.

Kafka — Durable, replayable event log. Lets us decouple services and add consumers (e.g. analytics) without changing producers.

React 19 + Vite + TS — Best-in-class DX, fastest HMR, type-safety end-to-end.

PWA + Workbox + Dexie — True offline-first — students in low-bandwidth areas can keep learning.

TanStack Query + Zustand — Purpose-built tools: server cache vs UI state, no Redux boilerplate.

Tailwind + CVA — Design-system friendly, tiny CSS bundle, no runtime.

Kubernetes — Industry standard, multi-cloud, autoscaling, self-healing.

Jenkins + Trivy + OWASP — Open, transparent, no vendor lock-in for CI/CD and security.

10. Roadmap & Future Enhancements

- AI tutor (LLM) for personalized doubt-solving (planned: Spring AI + LangChain4j).
- Mobile native app via React Native reusing the same REST + WS contracts.
- GraphQL gateway (Apollo Federation) for power-user mobile clients.
- Service mesh (Istio/Linkerd) for advanced mTLS and traffic shaping.
- Event-sourced read models with Kafka Streams / Apache Flink.
- Multi-region active-active deployment with cross-region Kafka MirrorMaker.

- Terraform-managed infrastructure (IaC beyond Kubernetes YAML).
- eBPF-based observability (Pixie / Cilium Tetragon).

— End of Technical Documentation —

Prepared for Smart India Hackathon 2026 by Team Acadevia.